

Spring AOP Pointcut Expressions — Quick Revision

1. What Is a Pointcut Expression?

Advice = what extra behaviour should run (logging, security, etc). Pointcut expression = **where** that advice should run. It just decides which method executions to intercept — it doesn't execute anything itself.

2. The `execution()` Designator

Most commonly used designator. Matches methods based on access modifier, return type, class, method name, parameters, exceptions. Essential part = return type + method name + parameters. Broadest form: `execution(* *(..))`.

2.1 Exact Method Matching

You can write a super-specific expression matching one exact method signature (modifier + return type + class + method name + params). Real apps usually use broader patterns instead.

3. Wildcards: `*` and `..`

`*` = matches anything in one position/segment (any return type, any method name, one package level). `..` = zero or more elements (any number of params, or zero-or-more package levels/subpackages). Combination gives patterns like `get*`, `*Student`, `*Student*`, `(*)` for exactly one param, `(..)` for any number.

4. Common `execution()` Patterns

Quick patterns to remember: match by method name everywhere, match every method, match only public methods, match all methods in a package+subpackages using `..*`. Table of common requirement→expression mappings is the cheat-sheet here.

5. The `within()` Designator

`execution()` checks method signature; `within()` checks only the class/package where method is declared. Can target exact class, direct classes in a package (`service.*`), or package + subpackages (`service..*`).

6. `execution()` vs `within()`

Both can achieve similar "match this layer" results, but `execution()` is more powerful (filters by modifier/return/name/params) while `within()` only restricts location. They're often combined with `&&` for precision (e.g. public getters only inside service layer).

7. The `@annotation()` Designator

Matches methods carrying a specific annotation (method-level). Useful for opt-in behaviour like auditing, execution-time tracking, authorization — instead of applying to a

whole package, only annotated methods get the aspect.

8. The bean() Designator

Matches using the **Spring bean name**, not the Java class name. Supports wildcards like `bean(*Service)` to match multiple bean names ending in "Service". Specific to Spring AOP.

9. Combining Pointcut Expressions

Use boolean operators: `&&` (AND — both must match), `||` (OR — either matches), `!` (NOT — must not match). Parentheses `()` help keep complex combined expressions readable.

10. Named Pointcuts with @Pointcut

Instead of repeating long expressions everywhere, define a reusable named pointcut with `@Pointcut("...")` on an empty method, then reference that method name (e.g. `serviceLayer()`) inside `@Before`, `@After` etc.

11. Centralizing Application Pointcuts

Put all your named pointcuts in one shared class (e.g. `ApplicationPointcuts`). Other aspect classes can reuse them via fully qualified name. Keeps layer definitions in one place — easy to maintain if packages change.

12. Advanced Designators — @within() and @target()

`@within(Annotation)` = matches methods **declared inside a class** carrying that annotation.
`@target(Annotation)` = matches based on the **runtime object's actual class** carrying the annotation. Difference shows up with inheritance: `@target` matches even inherited methods, `@within` doesn't (checks where method was declared).

13. args() and @args()

`args(Type)` = checks the **runtime argument type** passed into the method.
`@args(Annotation)` = checks if the runtime argument's class carries that annotation. Neither checks the annotation on the method parameter declaration itself.

14. this() and target()

Spring AOP uses a proxy in front of the real bean: Caller → Proxy → Actual bean.

`this(Type)` checks the **proxy's type**, `target(Type)` checks the **actual bean's type**. With interfaces, proxy type = interface, target type = concrete class.

15. JDK Proxy vs CGLIB Proxy

JDK proxy (interface-based): proxy only implements the interface, not the concrete class — so `this(Interface)` matches but `this(ConcreteClass)` normally doesn't. **CGLIB proxy** (class-based): proxy subclasses the actual class — so both `this()` and `target()` match the concrete class.

16. target() vs @target()

`target(Type)` checks the Java **type** of the actual bean. `@target(Annotation)` checks if that actual bean's class carries the given **annotation**. Different purpose — type-check vs annotation-check.

17. Configuring the Proxy Type

Spring Boot defaults to class-based (CGLIB) proxies (`spring.aop.proxy-target-class=true`). Can switch to JDK interface-based proxies by setting it to `false`. In plain Spring (no Boot), configure via `@EnableAspectJAutoProxy(proxyTargetClass = true/false)` — default there is `false`.

18. Quick Reference

Cheat table of all designators: `execution()` → method signature, `within()` → class/package, `@annotation()` → method annotation, `bean()` → Spring bean name, `@within()` → annotation on declaring type, `@target()` → annotation on runtime class, `args()` → runtime arg types, `@args()` → annotation on runtime arg types, `this()` → proxy type, `target()` → actual bean type.

19. Practical Guidelines

Prefer pointcuts tied to clear architectural boundaries (`within(package..*)`) over overly broad ones (`execution(* *(..))`). Use `execution()` for signature-specific matching, `within()` for whole layers, `@annotation()` for opt-in behaviour, named pointcuts to avoid repetition. Remember: AOP works through proxies, so a method calling another method **within the same class** bypasses the proxy — advice won't trigger.

20. Final Mental Model

A pointcut answers: "**Which method executions should get the aspect behaviour?**" Most important/commonly used designators: `execution()`, `within()`, `@annotation()`, `bean()`. The advanced ones (`this`, `target`, `args`, `@args`, `@within`, `@target`) matter mainly when dealing with inheritance, runtime types, or proxy internals.